

Lab Task 6

Course Title: Parallel and Distributed Computing (CS-426L)

Title Of Task

" Parallel Processing Using Java Fork/Join Framework and RecursiveTask"

SUBMITTED BY:

Name: Ibtehaj Ali Mirza

Student ID: 247200

Semester: 5th

Section: BS-CYS-F-24-**A**

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: 25 October, 2026

AIR UNIVERSITY, KHARIAN CAMPUS

DEPARTMENT OF COMPUTER SCIENCE

Question 1	CLO-1	GA-4	C2	Marks 1

Create a Java program named StudentMarks.java that uses a HashMap to store the names and marks of at least five students.

Your program must:

1. Insert five student records into the map.
2. Display all student records.
3. Update the marks of one student.
4. Retrieve and display the marks of a specific student.
5. Display the total number of students in the map.

The program should display all student records, the updated marks of one student, the marks of a specific student, and the total number of students.

Code:

```
import java.util.HashMap;
import java.util.Map;

public class StudentMarks {
    public static void main(String[] args) {
        HashMap<String, Integer> studentMarks = new HashMap<>();

        studentMarks.put("Ibtehaj", 85);
        studentMarks.put("Ali", 92);
        studentMarks.put("Asad", 78);
        studentMarks.put("Usama", 88);
        studentMarks.put("Haider", 65);

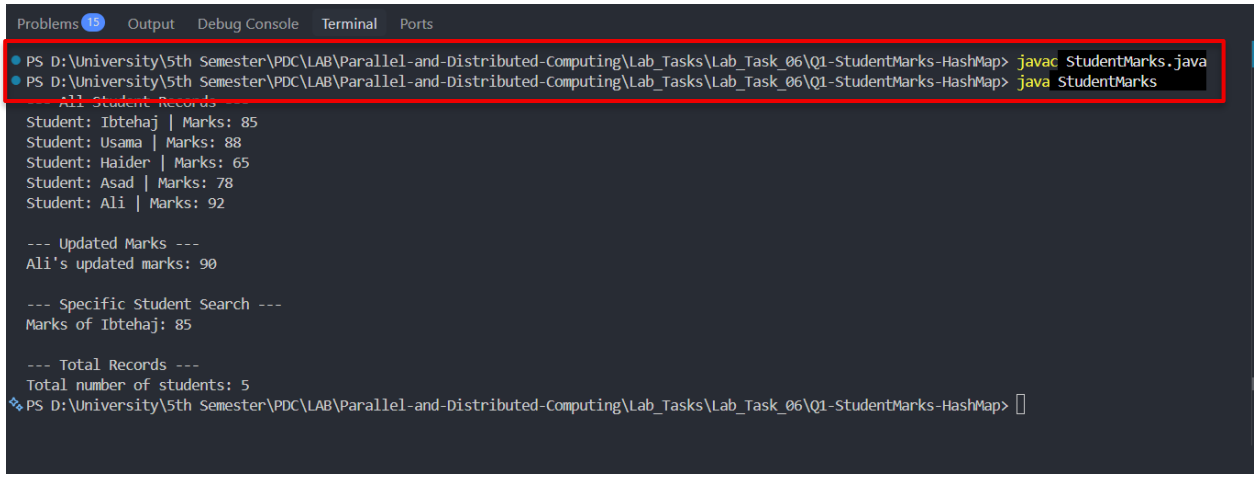
        System.out.println("--- All Student Records ---");
        for (Map.Entry<String, Integer> entry : studentMarks.entrySet()) {
            System.out.println("Student: " + entry.getKey() + " | Marks: " +
entry.getValue());
        }

        studentMarks.put("Ali", 90);
        System.out.println("\n--- Updated Marks ---");
        System.out.println("Ali's updated marks: " + studentMarks.get("Ali"));

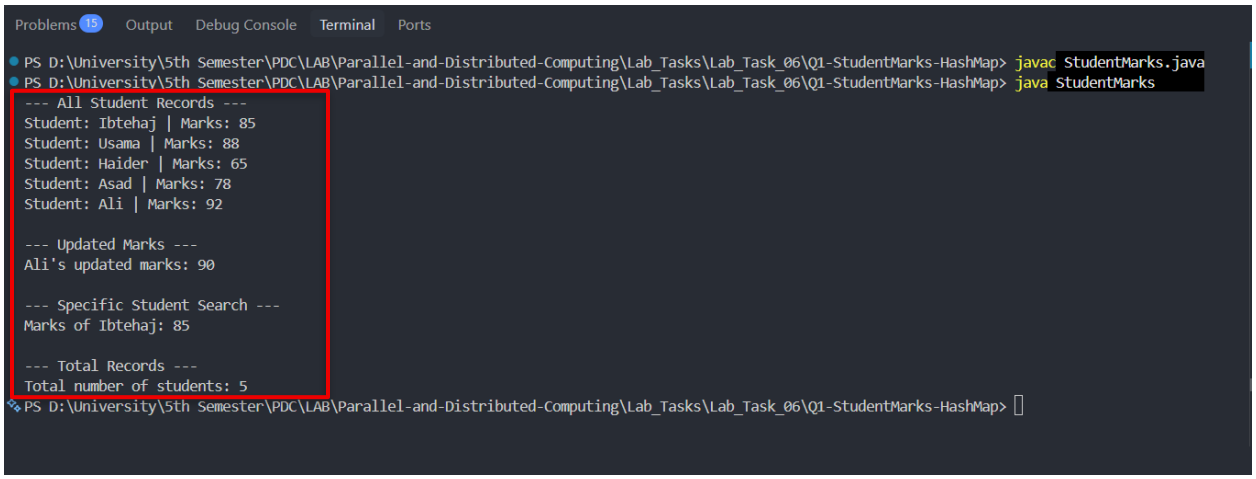
        String studentName = "Ibtehaj";
        System.out.println("\n--- Specific Student Search ---");
        System.out.println("Marks of " + studentName + ": " +
studentMarks.get(studentName));

        System.out.println("\n--- Total Records ---");
        System.out.println("Total number of students: " + studentMarks.size());
    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

This program uses a HashMap to store the names and marks of five students. First, it adds the records of 5 students to the map. Then, it displays all the student records using a loop. After displaying the records, the program updates Ali's marks from 85 to 90 using the put() method. It also uses the get() method to find and display Sara's marks. Finally, the size() method is used to show the total number of students stored in the map, which is five.

This program demonstrates how a HashMap can be used to store student records, update marks, search for a student's marks, and count the total records.

Question 2	CLO - 1	GA - 4	C2	Marks 1.5

Create a Java program named SynchronizedStudentMap.java that uses Collections.synchronizedMap() to create a synchronized map.
Youur program must:

1. Create a HashMap and convert it into a synchronized map.
2. Create an ExecutorService with three threads.
3. Make each thread insert 100 unique student records into the map.
4. Use shutdown() and awaitTermination() to ensure that all tasks finish before displaying the results.
5. Display the expected and actual number of entries.

Explain why a synchronized map is used when multiple threads access shared data. Also explain the purpose of shutdown() and awaitTermination().

Code:

```java
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class SynchronizedStudentMap {
    public static void main(String[] args) {
        Map<String, Integer> studentMap = Collections.synchronizedMap(new
HashMap<>());

        int threadCount = 3;
        ExecutorService executor = Executors.newFixedThreadPool(threadCount);

        for (int i = 0; i < threadCount; i++) {
            final int threadId = i;
            executor.execute(new Runnable() {
                @Override
                public void run() {
                    int start = (threadId * 100) + 1;
                    int end = start + 100;

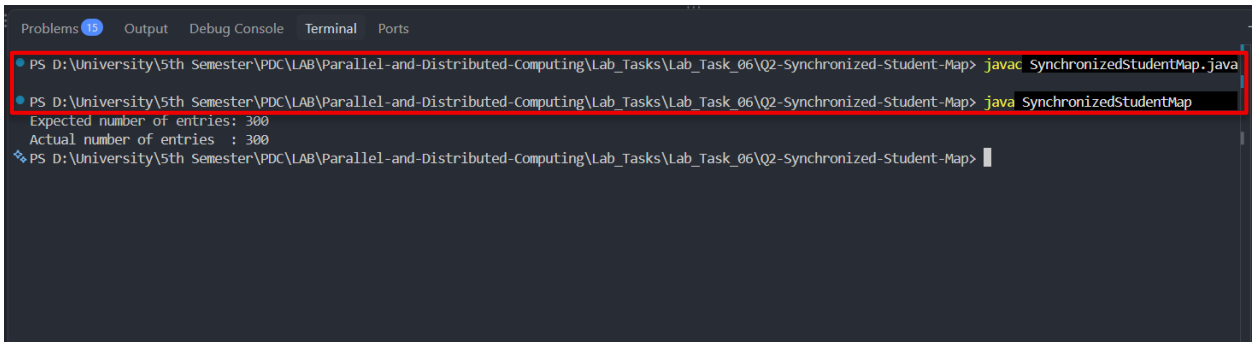
                    for (int j = start; j < end; j++) {
                        studentMap.put("Student_" + j, 70);
                    }
                }
            });
        }

        executor.shutdown();
        try {
```

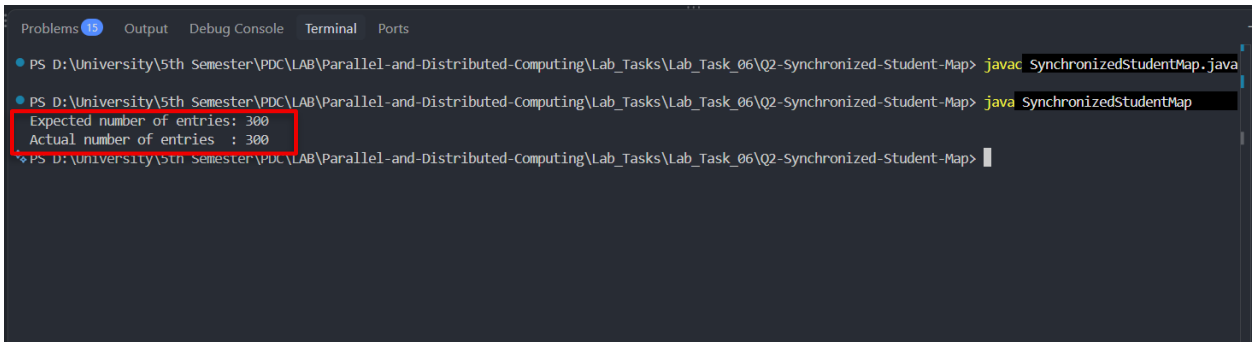
```
            executor.awaitTermination(5, TimeUnit.SECONDS);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        int expectedEntries = threadCount * 100;
        System.out.println("Expected number of entries: " + expectedEntries);
        System.out.println("Actual number of entries  : " + studentMap.size());
    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

This program uses a synchronized map to store student records safely when multiple threads are working at the same time. First, it creates a HashMap and converts it into a synchronized map using Collections.synchronizedMap().

An ExecutorService with three threads is created. Each thread inserts 100 unique student records into the map, making a total of 300 records. The shutdown() method stops the executor from accepting new tasks, while awaitTermination() waits for the tasks to finish.

At the end, the program displays the expected number of entries and the actual number of records stored in the map. If all tasks complete successfully, both values should be 300.

1. Why is a synchronized map used when multiple threads access shared data?

A synchronized map allows multiple threads to access and update the same map safely. It helps prevent conflicts when threads modify the map at the same time and keeps its individual operations synchronized.

2. What is the purpose of shutdown() and awaitTermination()?

- shutdown() stops the executor from accepting new tasks but allows previously submitted tasks to finish.
- awaitTermination() makes the main thread wait for the executor to finish its tasks, up to the specified time limit of five seconds.

Question 3	CLO - 1	GA - 4	C2	Marks 2.5

Create a Java program named ConcurrentStudentMap.java that uses ConcurrentHashMap to manage student records with multiple threads.

Your program must:

1. Create a ConcurrentHashMap.
2. Create an ExecutorService with five threads.
3. Make each thread insert 1,000 unique student records.
4. Retrieve the values associated with the inserted keys.
5. Measure the total execution time in milliseconds.
6. Display the number of threads, expected entries, actual entries, and execution time.

Explain why ConcurrentHashMap is suitable for concurrent access. Compare its purpose with that of a synchronized map and state why execution times may differ between computers.

Code:

```java
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ConcurrentStudentMap {
    public static void main(String[] args) {
        ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

        int threadCount = 5;
        int recordsPerThread = 1000;
        ExecutorService executor = Executors.newFixedThreadPool(threadCount);

        long startTime = System.currentTimeMillis();

        for (int i = 0; i < threadCount; i++) {
            final int threadId = i;
            executor.execute(new Runnable() {
```

```
                @Override
                public void run() {
                    int start = (threadId * recordsPerThread) + 1;
                    int end = start + recordsPerThread;

                    for (int j = start; j < end; j++) {
                        map.put("Student_" + j, 80);
                    }

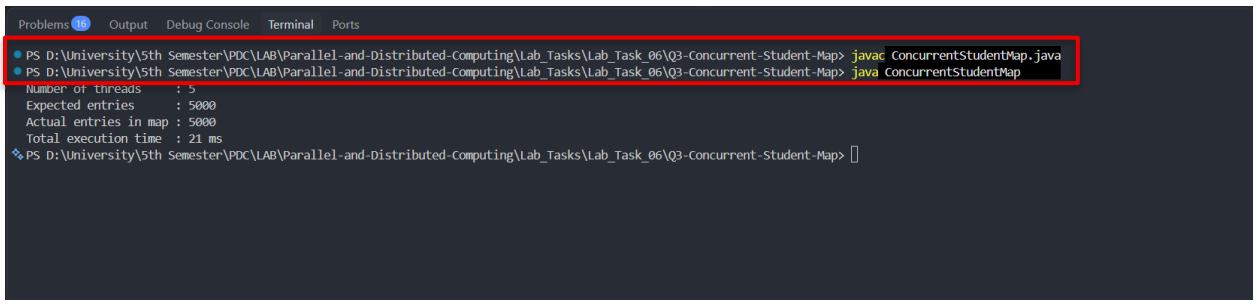
                    for (int j = start; j < end; j++) {
                        Integer marks = map.get("Student_" + j);
                    }
                }
            });
        }

        executor.shutdown();
        try {
            executor.awaitTermination(10, TimeUnit.SECONDS);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

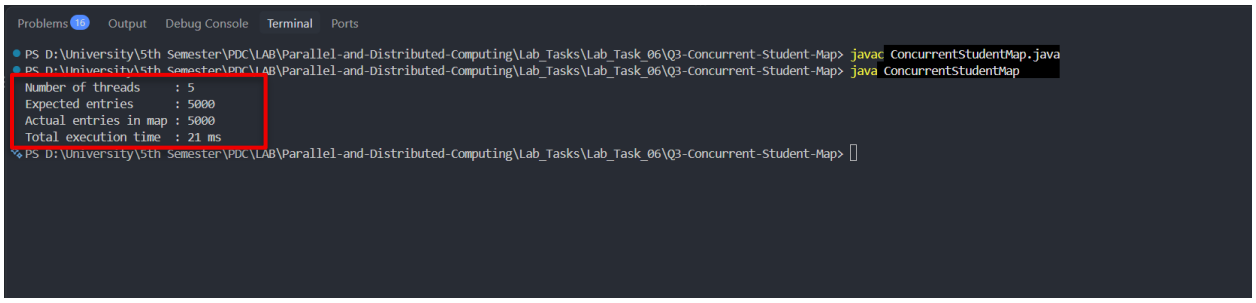
        long endTime = System.currentTimeMillis();
        long executionTime = endTime - startTime;

        int expectedEntries = threadCount * recordsPerThread;
        System.out.println("Number of threads     : " + threadCount);
        System.out.println("Expected entries      : " + expectedEntries);
        System.out.println("Actual entries in map : " + map.size());
        System.out.println("Total execution time  : " + executionTime + " ms");
    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

This program uses ConcurrentHashMap to store student records while multiple threads are working at the same time. An executor with five threads is created, and each thread inserts 1,000 unique student records into the map. Therefore, the expected total number of records is 5,000.

After inserting the records, each thread retrieves the marks associated with its student keys using the get() method. The program uses shutdown() and awaitTermination() to allow the submitted tasks to finish.

The execution time is measured in milliseconds using System.currentTimeMillis(). Finally, the program displays the number of threads, expected entries, actual entries, and total execution time. If all tasks complete successfully, the actual number of entries should be 5,000.

Q1. Why is ConcurrentHashMap suitable for concurrent access?

ConcurrentHashMap is designed to allow multiple threads to access and update a map safely. It supports concurrent operations without requiring the entire map to be locked for every operation, making it useful when several threads work with shared data.

Q2. Compare ConcurrentHashMap with a synchronized map.

A synchronized map uses synchronization to protect individual map operations, while ConcurrentHashMap is designed to support greater concurrency. A synchronized map can cause more threads to wait for access to the same map, whereas ConcurrentHashMap allows more operations to proceed concurrently. Both are useful when multiple threads share map data.

Q3. Why may execution times differ between computers?

Execution times may differ because computers have different processors, numbers of CPU cores, RAM, and system workloads. Java runtime performance and background processes can also affect the results. Therefore, the same program may take different amounts of time on different computers.

~ End ~